

IncidentFlow

Application Moderne et Dynamique de Gestion d'Incidents

**Guide Master de Préparation Technique à la
Soutenance
(Questions, Réponses & Justifications Architecturales)**

Anas HADDOU

Élève Ingénieur en Génie Informatique

Entreprise : Netmar

Encadrante : Sophie MARTIN (Responsable SSI)

20 juillet 2026

Table des matières

1	Fiche Synthétique et Présentation Globale	3
1.1	Présentation du Projet	3
1.2	Objectifs Métiers et Techniques	3
2	Architecture Technique et Justification des Choix	4
2.1	Architecture Globale N-Tiers	4
2.2	Justification des Choix Technologiques	4
2.2.1	Frontend : React 19, Vite et ReactFlow	4
2.2.2	Backend : Spring Boot 3 et Java 17	4
2.2.3	Bases de Données et Caching : PostgreSQL et Redis	4
3	Détail des Algorithmes Majeurs et du Code Source	6
3.1	Algorithme de Validation de Graphe DFS (Depth-First Search)	6
3.2	Moteur d'Escalade Automatique des SLA	7
3.3	Révocation Instantanée de Session via Redis Blacklist	7
4	Banque de Questions et Réponses pour la Soutenance	8
4.1	Questions d'Architecture et Conception	8
4.2	Questions de Sécurité et Système (SSI)	8
4.3	Questions de Performance et Scalabilité	8
5	Scénario de Démonstration en Direct (Live Demo)	10
6	Perspectives d'Évolution et Intégration d'IA	11

1 Fiche Synthétique et Présentation Globale

1.1 Présentation du Projet

IncidentFlow est une solution logicielle d'entreprise de nouvelle génération dédiée à la gestion, la déclaration, l'escalade et le suivi des incidents informatiques (ITSM / ITIL). Conçue au sein de l'entreprise **Netmar**, l'application résout la problématique majeure des outils du marché : la rigidité des cycles de vie d'incidents intégrés en dur dans le code source backend.

Caractéristique	Détail Technique / Fonctionnel
Intitulé du Projet	IncidentFlow — Plateforme d'ITSM adaptatif
Auteur	Anas HADDOU (Élève Ingénieur Génie Informatique)
Entreprise d'Accueil	Netmar
Encadrante Référente	Sophie MARTIN (Responsable SSI)
Frontend	React 19, Vite, ReactFlow, Lucide Icons, Vanilla CSS
Backend	Spring Boot 3, Java 17 (LTS), Spring Security, Spring Data JPA
Bases de données	PostgreSQL 15 (Persistance relationnelle), Redis 7 (Cache & Blacklist)
Sécurité IAM	Keycloak (OIDC / OAuth 2.0) + Mode Hybride dev-mock
Conteneurisation	Docker, Docker Compose, Multi-stage builds

TABLE 1 – Fiche technique du projet IncidentFlow

1.2 Objectifs Métiers et Techniques

- **Agilité Opérationnelle** : Donner aux administrateurs la possibilité de modifier graphiquement les règles de transition et d'états d'un incident sans recompilation ni interruption de service (Zero-Downtime).
- **Conformité SLA** : Assurer le respect des contrats d'engagement de service grâce à un planificateur d'escalade automatique en arrière-plan.
- **Sécurité et Traçabilité** : Garantir un contrôle d'accès basé sur les rôles (RBAC), la traçabilité intégrale de chaque modification (Audit Trail) et la révocation instantanée de session.

2 Architecture Technique et Justification des Choix

2.1 Architecture Globale N-Tiers

L'architecture d'IncidentFlow est structurée en couches étanches :

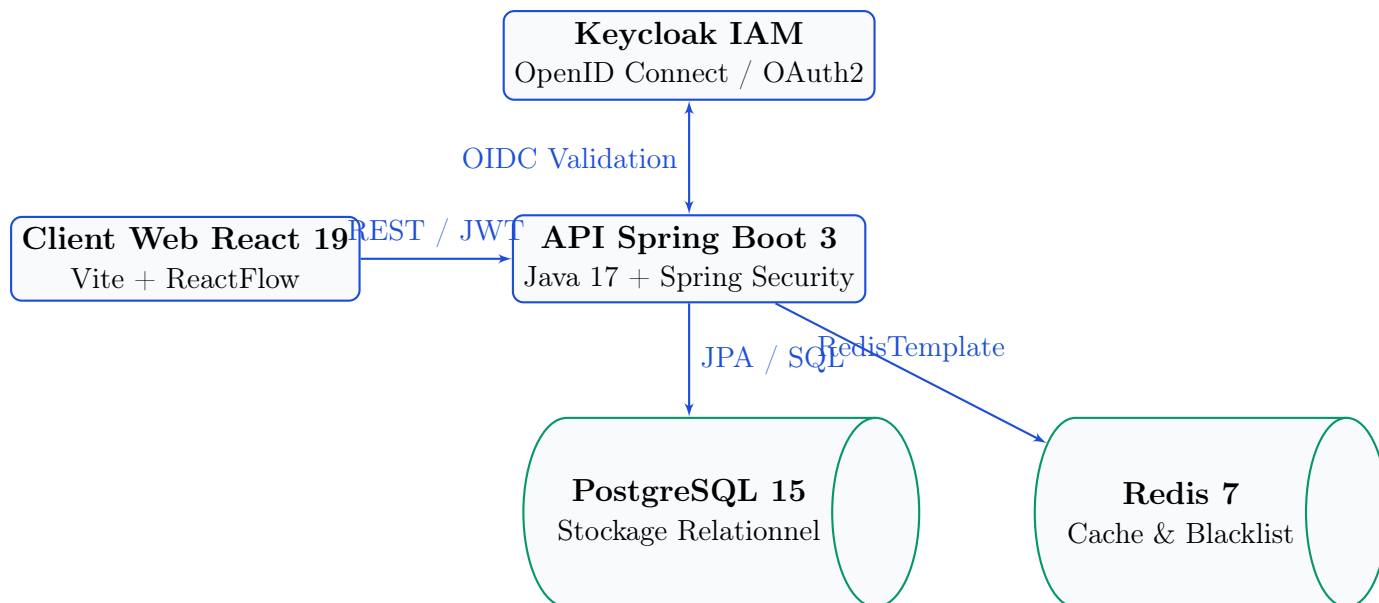


FIGURE 1 – Schéma de l'architecture distribuée IncidentFlow

2.2 Justification des Choix Technologiques

2.2.1 Frontend : React 19, Vite et ReactFlow

- **React 19 & Vite** : Vite offre un démarrage immédiat en environnement de développement grâce aux modules ES natifs du navigateur, ainsi qu'un temps de compilation extrêmement court en comparaison avec les bundlers traditionnels.
- **ReactFlow** : Bibliothèque spécialisée pour le rendu dynamique de graphes interactifs. Elle permet d'afficher et d'éditer les workflows sous forme de nœuds et d'interconnexions personnalisables par glisser-déposer.

2.2.2 Backend : Spring Boot 3 et Java 17

- **Java 17 (LTS)** : Version à support long terme offrant des gains d'exécution importants, des structures immutables (*Records*) et un garbage collector optimisé (G1/ZGC).
- **Spring Boot 3** : Écosystème standard de l'industrie pour les applications critiques d'entreprise, offrant une gestion déclarative des transactions (@Transactional), l'injection de dépendances et la sécurisation native via Spring Security 6.

2.2.3 Bases de Données et Caching : PostgreSQL et Redis

- **PostgreSQL 15** : Base relationnelle garantie ACID. Indispensable pour préserver les contraintes d'intégrité référentielle entre les incidents, les utilisateurs et les transitions de workflow.

- **Redis 7** : Stockage en mémoire utilisé d'une part pour le cache distribué (`@Cacheable`) des structures de workflow, et d'autre part pour la révocation instantanée de session.

3 Détail des Algorithmes Majeurs et du Code Source

3.1 Algorithme de Validation de Graphe DFS (Depth-First Search)

Afin d'empêcher la publication d'un workflow incohérent (contenant des états inaccessibles ou des pièges sans sortie), la méthode `validateWorkflowGraph()` du service `WorkflowService` applique un algorithme fondé sur un double parcours en profondeur (DFS) :

Listing 1 – Algorithme de validation graphique par parcours DFS

```

1 public void validateWorkflowGraph(Workflow workflow) {
2     if (workflow.getStates() == null || workflow.getStates().isEmpty())
3         ↪ {
4         throw new IllegalArgumentException("Le workflow doit contenir au
5         ↪ moins un état.");
6     }
7
8     // 1. Validation de la présence des états obligatoires
9     boolean hasNouveau = workflow.getStates().stream().anyMatch(s -> s.
10     ↪ getName().equalsIgnoreCase("Nouveau"));
11     boolean hasCloture = workflow.getStates().stream().anyMatch(s -> s.
12     ↪ getName().equalsIgnoreCase("Cloture"));
13
14     if (!hasNouveau) throw new IllegalArgumentException("L'état initial
15     ↪ 'Nouveau' est obligatoire.");
16     if (!hasCloture) throw new IllegalArgumentException("L'état final '
17     ↪ Cloture' est obligatoire.");
18
19     // 2. Construction des listes d'adjacence directe et inverse
20     Map<String, List<String>> adj = new HashMap<>();
21     Map<String, List<String>> revAdj = new HashMap<>();
22
23     for (String state : stateNames) {
24         adj.put(state, new ArrayList<>());
25         revAdj.put(state, new ArrayList<>());
26     }
27
28     for (WorkflowTransition t : workflow.getTransitions()) {
29         String from = t.getFromState().trim().toLowerCase();
30         String to = t.getToState().trim().toLowerCase();
31         if (adj.containsKey(from) && adj.containsKey(to)) {
32             adj.get(from).add(to);
33             revAdj.get(to).add(from); // Graphe inverse
34         }
35     }
36
37     // 3. Detection des états orphelins (Forward DFS depuis 'Nouveau')
38     Set<String> visitedFromStart = new HashSet<>();
39     dfs("nouveau", adj, visitedFromStart);
40     List<String> orphans = stateNames.stream().filter(name -> !
41     ↪ visitedFromStart.contains(name)).toList();
42     if (!orphans.isEmpty()) {
43         throw new IllegalArgumentException("Etats orphelins detectes : "
44         ↪ + String.join(", ", orphans));
45     }
46
47     // 4. Detection des impasses (Reverse DFS depuis 'Cloture')

```

```
40 Set<String> visitedFromEnd = new HashSet<>();
41 dfs("cloture", revAdj, visitedFromEnd);
42 List<String> deadlocks = stateNames.stream().filter(name -> !
    ↪ visitedFromEnd.contains(name)).toList();
43 if (!deadlocks.isEmpty()) {
44     throw new IllegalArgumentException("Impasses detectees : " +
    ↪ String.join(", ", deadlocks));
45 }
46 }
```

3.2 Moteur d'Escalade Automatique des SLA

Le service `EscalationService` exécute un job en arrière-plan annoté avec `@Scheduled(fixedRate = 15000)`. Il scrute les incidents actifs dont la date d'échéance `slaDueAt` est dépassée ou critique, augmente leur priorité à `Critical`, les réassigne automatiquement aux responsables et consigne l'événement dans le journal d'audit.

3.3 Révocation Instantanée de Session via Redis Blacklist

Pour surmonter le problème de la validité persistante des jetons JWT après le bannissement d'un utilisateur, un filtre personnalisé `SessionValidationFilter` intercepte chaque requête HTTP. Il vérifie l'existence de la clé `blacklist:user:{email}` dans Redis en $O(1)$ et renvoie immédiatement un statut 403 `Forbidden` si la session a été révoquée par l'administrateur.

4 Banque de Questions et Réponses pour la Soutenance

Cette section rassemble les questions techniques les plus probables posées par l'encadrante **Sophie MARTIN** ou le jury, assorties des réponses stratégiques recommandées.

4.1 Questions d'Architecture et Conception

Q1 : Pourquoi ne pas avoir retenu une solution BPMN éprouvée du marché comme Camunda ou Activiti ?

Réponse : Camunda est une suite BPMN 2.0 complète mais très lourde à déployer et nécessitant des compétences poussées. Pour les besoins de Netmar, nous recherchions une solution d'ITSM agile, directement intégrée à l'UI React via ReactFlow, sans ajouter une dépendance d'infrastructure volumineuse ni imposer une courbe d'apprentissage complexe aux équipes métiers.

Q2 : Comment le système gère-t-il la modification d'un workflow alors que des incidents y sont rattachés ?

Réponse : Nous appliquons un schéma de **versionnage immuable**. Lorsqu'un workflow lié à des incidents existants est modifié, sa version actuelle passe à `active = false` et conserve son intégrité. Une nouvelle version (v2, v3, etc.) est instanciée pour recevoir les futurs incidents. Ainsi, l'historique et la traçabilité des anciens incidents ne sont jamais corrompus.

4.2 Questions de Sécurité et Système (SSI)

Q3 : Comment assurez-vous la révocation immédiate d'un jeton JWT en cas de compromission d'un compte ?

Réponse : Les jetons JWT étant autonomes et sans état, nous les couplons avec une liste noire stockée dans **Redis**. Notre filtre Spring Security (`SessionValidationFilter`) contrôle chaque requête en interrogeant la clé `blacklist:user:{email}`. Si l'utilisateur est révoqué dans Keycloak ou banni par le responsable sécurité, le filtre retourne instantanément un statut 403 Forbidden.

Q4 : Quelle est votre protection contre les failles courantes comme les injections SQL ou XSS ?

Réponse :

- **SQL Injection :** Nous utilisons Spring Data JPA et Hibernate, qui exécutent des requêtes préparées avec paramètres typés, éliminant tout risque d'injection SQL.
- **XSS :** React échappe par défaut le contenu rendu dans le DOM. Les commentaires formatés en Markdown sont également assainis côté client avant affichage.

4.3 Questions de Performance et Scalabilité

Q5 : Le planificateur d'escalade SLA qui s'exécute toutes les 15 secondes n'impacte-t-il pas la base de données ?

Réponse : Non, car la requête cible uniquement les incidents actifs (`status NOT IN ('Résolu', 'Clôturé')`) et s'appuie sur des index SQL posés sur les colonnes `status` et `sla_due_at`. Le volume scruté reste donc très faible et l'exécution s'effectue en quelques millisecondes.

5 Scénario de Démonstration en Direct (Live Demo)

Pour présenter efficacement la solution en 10 minutes lors de la réunion :

- Étape 1 : Authentification Opérateur (2 min) :** Connectez-vous avec le compte `marie.1@netmar.com`. Déclarez un nouvel incident de catégorie *Informatique* intitulé "*Panne du serveur de base de données*". Montrez la génération automatique du code `INC-2026-xxx` et de la date limite SLA.
- Étape 2 : Édition Graphique de Workflow (3 min) :** Déconnectez-vous et connectez-vous avec le compte Administrateur `anas@netmar.com`. Accédez au concepteur React-Flow. Ajoutez un état intermédiaire (ex : *Expertise Niveau 3*) et reliez les transitions. Sauvegardez et montrez la notification du validateur DFS confirmant l'absence d'impasses.
- Étape 3 : Démonstration de l'Escalade SLA (3 min) :** Ouvrez la console backend Spring Boot pour montrer les messages du planificateur `[PLANIFICATEUR]`. Montrez le basculement automatique d'un incident en priorité `Critical` avec réassignation au responsable et ajout du commentaire d'alerte automatique.
- Étape 4 : Audit Trail et Traçabilité (2 min) :** Ouvrez la vue détaillée de l'incident et parcourez l'historique complet pour démontrer que chaque changement d'état et d'assignation est consigné avec précision.

6 Perspectives d'Évolution et Intégration d'IA

Comme développé dans notre rapport spécifique d'intégration de l'Intelligence Artificielle, les évolutions futures d'IncidentFlow prévoient :

1. **Classification et Tri Automatique par LLM** : Analyse du texte explicatif lors de la saisie d'un incident afin de suggérer automatiquement la catégorie appropriée et le workflow correspondant.
2. **Estimation Automatique de la Sévérité** : Utilisation de modèles NLP pour évaluer la gravité réelle à partir des mots-clés d'urgence et du contexte utilisateur.
3. **Assistant de Résolution RAG (Retrieval-Augmented Generation)** : Interrogation de la base vectorielle des incidents passés résolus pour proposer immédiatement des pistes de résolution aux techniciens du support.

Document rédigé pour la préparation de soutenance de l'application
IncidentFlow.